11 Switch Expressions and Statements

You can use the `switch` keyword as either a statement or an expression. Like all expressions, `switch` expressions evaluate to a single value and can be used in statements. Switch expressions may contain "`case L ->`" labels that eliminate the need for `break` statements to prevent fall through. You can use a `yield` statement to specify the value of a switch expression.

For background information about the design of `switch` expressions, see JEP 361.

Topics

- Arrow Cases
- Colon Cases and the the yield Statement
- Qualified enum Constants as case Constants
- Primitive Values in switch Expressions and Statements
- Exhaustiveness of switch
- Completion and switch Expressions

Arrow Cases

Consider the following `switch` statement that prints the number of letters of a day of the week:

COPY

```
public enum Day { SUNDAY, MONDAY, TUESDAY,
    WEDNESDAY, THURSDAY, FRIDAY, SATURDAY; }
// ...

int numLetters = 0;
Day day = Day.WEDNESDAY;
switch (day) {
    case MONDAY:
    case FRIDAY:
    case SUNDAY:
       numLetters = 6;
        break;
    case TUESDAY:
        numLetters = 7;
        break;
    case THURSDAY:
    case SATURDAY:
        numLetters = 8;
        break;
    case WEDNESDAY:
        numLetters = 9;
        break;
}
System.out.println(numLetters);
```

It would be better if you could "return" the length of the day's name instead of storing it in the variable `numLetters`; you can do this with a `switch` expression. Furthermore, it would be better if you didn't need `break` statements to prevent fall through; they are laborious to write and easy to forget. You can do this with an *arrow case*. The following is a `switch` expression that uses arrow cases to print the number of letters of a day of the week:

COPY

```
Day day = Day.WEDNESDAY;
System.out.println(
    switch (day) {
        case MONDAY, FRIDAY, SUNDAY -> 6;
        case TUESDAY                -> 7;
        case THURSDAY, SATURDAY     -> 8;
        case WEDNESDAY              -> 9;
    }
);
```

An arrow case has the following form:

```
case label_1, label_2, ..., label_n -> expression;|throw-statement;|block
```

When the Java runtime matches any of the labels to the left of the arrow, it runs the code to the right of the arrow and does not fall through; it does not run any other code in the `switch` expression (or statement). If the code to the right of the arrow is an expression, then the value of that expression is the value of the `switch` expression.

You can use arrow cases in `switch` statements. The following is like the first example, except it uses arrow cases instead of colon cases:

```
int numLetters = 0;
Day day = Day.WEDNESDAY;
switch (day) {
    case MONDAY, FRIDAY, SUNDAY -> numLetters = 6;
    case TUESDAY                -> numLetters = 7;
    case THURSDAY, SATURDAY     -> numLetters = 8;
    case WEDNESDAY              -> numLetters = 9;
};
System.out.println(numLetters);
```

An arrow case along with its code to its right is called a `switch`-labeled rule.

Colon Cases and the the yield Statement

A *colon case* is a `case` label in the form `case L:`. You can use colon cases in `switch` expressions. A colon case along with its code to the right is called a `switch`-labeled statement group:

```
Day day = Day.WEDNESDAY;
int numLetters = switch (day) {
    case MONDAY:
    case FRIDAY:
    case SUNDAY:
        System.out.println(6);
        yield 6;
    case TUESDAY:
        System.out.println(7);
        yield 7;
    case THURSDAY:
    case SATURDAY:
        System.out.println(8);
        yield 8;
    case WEDNESDAY:
        System.out.println(9);
        yield 9;
};
System.out.println(numLetters);
```

The previous example uses `yield` statements. They take one argument, which is the value that the colon case produces in a `switch` expression.

The `yield` statement makes it easier for you to differentiate between `switch` statements and `switch` expressions. A `switch` statement, but not a `switch` expression, can be the target of a `break` statement. Conversely, a `switch` expression, but not a `switch` statement, can be the target of a `yield` statement.

Note

It's recommended that you use arrow cases. It's easy to forget to insert `break` or `yield` statements when using colon cases; if you do, you might introduce unintentional fall through in your code.

For arrow cases, to specify multiple statements or code that are not expressions or `throw` statements, enclose them in a block. Specify the value that the arrow case produces with the `yield` statement:

COPY

```
int numLetters = switch (day) {
    case MONDAY, FRIDAY, SUNDAY -> {
        System.out.println(6);
        yield 6;
    }
    case TUESDAY -> {
        System.out.println(7);
        yield 7;
    }
    case THURSDAY, SATURDAY -> {
        System.out.println(8);
        yield 8;
    }
    case WEDNESDAY -> {
        System.out.println(9);
        yield 9;
    }
};
```

Qualified enum Constants as case Constants

You can use qualified `enum` constants as `case` constants in `switch` expressions and statements.

Consider the following `switch` expression whose selector expression is an `enum` type:

COPY

```
public enum Standard { SPADE, HEART, DIAMOND, CLUB }
// ...

static void determineSuitStandardDeck(Standard d) {
    switch (d) {
        case SPADE   -> System.out.println("Spades");
        case HEART   -> System.out.println("Hearts");
        case DIAMOND -> System.out.println("Diamonds");
        default      -> System.out.println("Clubs");
    }
}
```

In the following example, the type of the selector expression is an interface that's been implemented by two `enum` types. Because the type of the selector expression isn't an `enum` type, this `switch` expression uses guarded patterns instead:

COPY

```
sealed interface CardClassification permits Standard, Tarot {}
public enum Standard implements CardClassification
    { SPADE, HEART, DIAMOND, CLUB }
public enum Tarot implements CardClassification
    { SPADE, HEART, DIAMOND, CLUB, TRUMP, EXCUSE }
// ...

static void determineSuit(CardClassification c) {
    switch (c) {
        case Standard s when s == Standard.SPADE    -> System.out.println("Spades");
        case Standard s when s == Standard.HEART    -> System.out.println("Hearts");
        case Standard s when s == Standard.DIAMOND  -> System.out.println("Diamonds");
        case Standard s                             -> System.out.println("Clubs");
        case Tarot t when t == Tarot.SPADE          -> System.out.println("Spades or Piques");
        case Tarot t when t == Tarot.HEART          -> System.out.println("Hearts or C\u0153ur");
        case Tarot t when t == Tarot.DIAMOND        -> System.out.println("Diamonds or Carreaux");
        case Tarot t when t == Tarot.CLUB           -> System.out.println("Clubs or Trefles");
        case Tarot t when t == Tarot.TRUMP          -> System.out.println("Trumps or Atouts");
        case Tarot t                                -> System.out.println("The Fool or L'Excuse");
    }
}
```

However, `switch` expressions and statements allow qualified `enum` constants, so you could rewrite this example as follows:

COPY

```
static void determineSuitQualifiedNames(CardClassification c) {
    switch (c) {
        case Standard.SPADE   -> System.out.println("Spades");
        case Standard.HEART   -> System.out.println("Hearts");
        case Standard.DIAMOND -> System.out.println("Diamonds");
        case Standard.CLUB    -> System.out.println("Clubs");
        case Tarot.SPADE      -> System.out.println("Spades or Piques");
        case Tarot.HEART      -> System.out.println("Hearts or C\u0153ur");
        case Tarot.DIAMOND    -> System.out.println("Diamonds or Carreaux");
        case Tarot.CLUB       -> System.out.println("Clubs or Trefles");
        case Tarot.TRUMP      -> System.out.println("Trumps or Atouts");
        case Tarot.EXCUSE     -> System.out.println("The Fool or L'Excuse");
    }
}
```

Therefore, you can use an `enum` constant when the type of the selector expression is not an `enum` type provided that the `enum` constant's name is qualified and its value is assignment-compatible with the type of the selector expression.

Primitive Values in switch Expressions and Statements

In addition to the primitive types `char`, `byte`, `short`, and `int`, a `switch` expression or statement's selector expression can be of type `long`, `float`, `double`, and `boolean`, as well as the corresponding boxed types.

> **Note**
>
> This is a preview feature. A preview feature is a feature whose design, specification, and implementation are complete, but is not permanent. A preview feature may exist in a different form or not at all in future Java SE releases. To compile and run code that contains preview features, you must specify additional command-line options. See Preview Language and VM Features.
>
> See JEP 507: Primitive Types in Patterns, instanceof, and switch (Third Preview) for additional information.

If a selector expression is of type `long`, `float`, `double`, and `boolean`, then its case labels must have the same type as the selector expression or its corresponding boxed type. For example:

COPY

```
void whichFloat(float v) {
    switch (v) {
        case 0f ->
            System.out.println("Zero");
        case float x when x > 0f && x <= 10f ->
            System.out.println(x + " is between 1 and 10");
        case float x ->
            System.out.println(x + " is larger than 10");
    }
}
```

If you change the case label `0f` to `0`, you would get the following compile-time error:

```
error: constant label of type int is not compatible with switch selector type float
```

You can't use two floating-point literals as `case` labels that are *representationally equivalent*. (See the JavaDoc API documentation for the `Double` class for more information about representation equivalence.) The following example generates a compiler error. The value `0.999999999f` is representationally equivalent to `1.0f`:

```
void duplicateLabels(float v) {
    switch (v) {
        case 1.0f -> System.out.println("One");
        // error: duplicate case label
        case 0.999999999f -> System.out.println("Almost one");
        default -> System.out.println("Another float value");
    }
}
```

Switching on `boolean` values is a useful alternative to the ternary conditional operator (`?:`) because a `boolean` `switch` expression or statement can contain statements as well as expressions. For example, the following code uses a `boolean` `switch` expression to perform some logging when `false`:

```
long startProcessing(OrderStatus.NEW, switch (user.isLoggedIn()) {
    case true  -> user.id();
    case false -> { log("Unrecognized user"); yield -1L; }
});
```

Exhaustiveness of switch

The cases of a `switch` expression or statement must be *exhaustive*, which means that for all possible values, there must be a matching `case` label. Thus, a `switch` expression or statement normally require a `default` clause. However, for an `enum` `switch` expression that covers all known constants, the compiler inserts an implicit `default` clause, like the examples at the beginning of this section that print the number of letters in name of a day of the week.

The cases of a `switch` statement must be exhaustive if it uses pattern or `null` labels. See Pattern Matching with switch and Null case Labels for more information.

The following `switch` expression is not exhaustive and generates a compile-time error. The type coverage of its labels consist of the subtypes of `String` and `Integer`. However, it doesn't include the type of the selector expression, `Object`:

```
static int coverage(Object obj) {
    return switch (obj) {          // Error - not exhaustive
        case String s  -> s.length();
        case Integer i -> i;
    };
}
```

However, the type coverage of the case label `default` is all types, so the following example compiles:

```
static int coverage(Object obj) {
    return switch (obj) {
        case String s  -> s.length();
        case Integer i -> i;
        default        -> 0;
    };
}
```

Consequently, for a `switch` expression or statement to be exhaustive, the type coverage of its labels must include the type of the selector expression.

The compiler takes into account whether the type of a selector expression is a sealed class. The following `switch` expression compiles. It doesn't need a `default` case label because its type coverage is the classes `A`, `B`, and `C`, which are the only permitted subclasses of `S`, the type of the selector expression:

```
sealed interface S permits A, B, C { }
final class A implements S { }
final class B implements S { }
record C(int i) implements S { }  // Implicitly final
//...

    static int testSealedCoverage(S s) {
        return switch (s) {
            case A a -> 1;
            case B b -> 2;
            case C c -> 3;
        };
    }
```

The compiler can also determine the type coverage of a `switch` expression or statement if the type of its selector expression is a generic sealed class. The following example compiles. The only permitted subclasses of interface `I` are classes `A` and `B`. However, because the selector expression is of type `I<Integer>`, the `switch` block requires only class `B` in its type coverage to be exhaustive:

```
    sealed interface I<T> permits A, B {}
    final class A<X> implements I<String> {}
    final class B<Y> implements I<Y> {}

    static int testGenericSealedExhaustive(I<Integer> i) {
        return switch (i) {
        // Exhaustive as no A case possible!
            case B<Integer> bi -> 42;
        };
    }
```

The type of a `switch` expression or statement's selector expression can also be a generic record. As always, a `switch` expression or statement must be exhaustive. The following example doesn't compile. No match for a `Pair` exists that contains two values, both of type `A`:

```
record Pair<T>(T x, T y) {}
class A {}
class B extends A {}

static void notExhaustive(Pair<A> p) {
    switch (p) {
        // error: the switch statement does not cover all possible input values
        case Pair<A>(A a, B b) -> System.out.println("Pair<A>(A a, B b)");
        case Pair<A>(B b, A a) -> System.out.println("Pair<A>(B b, A a)");
    }
}
```

The following example compiles. Interface `I` is sealed. Types `C` and `D` cover all possible instances:

COPY

```
record Pair<T>(T x, T y) {}
sealed interface I permits C, D {}
record C(String s) implements I {}
record D(String s) implements I {}
// ...

static void exhaustiveSwitch(Pair<I> p) {
    switch (p) {
        case Pair<I>(I i, C c) -> System.out.println("C = " + c.s());
        case Pair<I>(I i, D d) -> System.out.println("D = " + d.s());
    }
}
```

If a `switch` expression or statement is exhaustive at compile time but *not* at run time, then a `MatchException` is thrown. This can happen when a class that contains an exhaustive `switch` expression or statement has been compiled, but a sealed hierarchy that is used in the analysis of the `switch` expression or statement has been subsequently changed and recompiled. Such changes are *migration incompatible* and may lead to a `MatchException` being thrown when running the `switch` statement or expression. Consequently, you need to recompile the class containing the `switch` expression or statement.

Consider the following class `PrintA` and sealed interface `OnlyAB`:

COPY

```
class PrintA {
    public static void main(String[] args) {
        System.out.println(switch (OnlyAB.getAValue()) {
            case A a -> 1;
            case B b -> 2;
        });
    }
}
```

COPY

```
sealed interface OnlyAB permits A, B {
    static OnlyAB getAValue() {
        return new A();
    }
}
final class A implements OnlyAB {}
final class B implements OnlyAB {}
```

The `switch` expression in the class `PrintA` is exhaustive and this example compiles. When you run `PrintA`, it prints the value `1`. However, suppose you edit `OnlyAB` as follows and compile this interface and *not* `PrintA`:

COPY

```
sealed interface OnlyAB permits A, B, C {
    static OnlyAB getAValue() {
        return new A();
    }
}
final class A implements OnlyAB {}
final class B implements OnlyAB {}
final class C implements OnlyAB {}
```

When you run `PrintA`, it throws a `MatchException`:

```
Exception in thread "main" java.lang.MatchException
            at PrintA.main(PrintA.java:3)
```

Completion and switch Expressions

A `switch` expression must either complete normally with a value or complete abruptly by throwing an exception

A. For example, the following code doesn't compile because the `switch` labeled rule doesn't contain a `yield` statement:

```
int i = switch (day) {
    case MONDAY -> {
        System.out.println("Monday");
        // error: block doesn't contain a yield statement
    }
    default -> 1;
};
```

The following example doesn't compile because the `switch` labeled statement group doesn't contain a `yield` statement:

```
i = switch (day) {
    case MONDAY, TUESDAY, WEDNESDAY:
        yield 0;
    default:
        System.out.println("Second half of the week");
        // error: group doesn't contain a yield statement
};
```

Because a `switch` expression must evaluate to a single value (or throw an exception), you can't jump through a `switch` expression with a `break`, `yield`, `return`, or `continue` statement, like in the following example:

```
z:
    for (int i = 0; i < MAX_VALUE; ++i) {
        int k = switch (e) {
            case 0:
                yield 1;
            case 1:
                yield 2;
            default:
                continue z;
                // error: illegal jump through a switch expression
        };
    // ...
    }
```